

PROCESSING 2D IMAGE TO 3D SPATIAL TERRAIN MESH

-Garima Singh

- **OBJECTIVE :**

Develop an image processing script that analyzes a flat 2D image to calculate and generate a normal/height map, and mathematically translate that map into a 3D spatial terrain mesh.

- **ROAD MAP:**

Phase 1: Environment Setup & Image Ingestion

- **Select Your Frameworks**
 - Set up a environment with core mathematical and rendering packages: NumPy (matrix operations), OpenCV (image manipulation), and Open3D or Trimesh (3D mesh handling and visualization).
- **Image Preprocessing**
 - Load your input image and convert it to **grayscale**. In a height map, pixel intensity directly corresponds to elevation.
 - Apply a mild **Gaussian Blur** to smooth out harsh pixel noise. High-frequency digital noise in a 2D image translates into jagged, unnatural spikes in a 3D mesh.

Phase 2: Generating the Elevation Maps

- **Height Map Normalization**
 - Convert your grayscale image array into a floating-point matrix scaled between . This gives a normalized baseline for Z-axis (elevation).

- **Normal Map Calculation**
 - To make 3D mesh interact naturally with lighting later, we need surface normal vectors. Calculate the horizontal and vertical image gradients using **filters**.
 - Compute the surface normal vector for every pixel
 - Normalize these vectors to unit length and map their components color space to generate and save your standard RGB Normal Map.

Phase 3: Mathematical Translation to a 3D Mesh

- **Vertex Generation**
 - Create an empty grid matching the height and width dimensions of your image.
 - Map each pixel location (i, j) to 3D space:
 - Flatten this grid into a 3d array representing the 3D vertices.
 -
- **Face Triangulation (Connecting the Dots)**
 - A 3D mesh cannot render without faces (polygons). Loop through the grid indices and divide each 2d pixel quad into two distinct triangles.
 - Define the clockwise or counter-clockwise vertex index order for every triangle to establish correct **face normals**.

Phase 4: Texturing, Export, and Real-Time Visualization

- **Texture Mapping (UV Coordinates)**
 - Generate a UV coordinate map that binds each 3D vertex back to its original relative position in the 2D image space. This allows to drape the original color image over newly generated 3D hills and valleys.
- **Mesh Compilation & Rendering**
 - Pass your arrays (Vertices, Faces, UVs, and Normals) into Open3D
 - Initialize an interactive camera window to rotate, pan, and inspect your generated 3D landscape.

- **Exporters**

- Write a utility function to package your calculated arrays into standard 3D file formats like **.OBJ** or **.PLY** so your custom terrain can be imported into external 3D software

Starting with project (draft 1):

Scope and Constraints

The pipeline is intentionally constrained to single-image, single-scale processing. Multi-scale octave blending, machine-learning depth estimation, and photogrammetric stereo reconstruction are explicitly out of scope, though they represent natural extension directions

Luminance as Elevation Proxy

A greyscale luminance value is treated as a proxy for surface elevation: bright regions correspond to high terrain, dark regions to low terrain. This assumption holds well for illuminated height-field photographs, satellite imagery with strong sun angles, and procedurally generated height textures. For artistic inputs, it provides a perceptually intuitive mapping.

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Gaussian Pre-smoothing

Raw images contain high-frequency luminance variation (sensor noise, texture detail, compression artefacts) that, if allowed to propagate, produces unphysically spiky terrain. A 2D isotropic Gaussian kernel with standard deviation σ is convolved with the luminance image prior to any geometric computation:

$$G(x, y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2)$$

Normal Vector Synthesis via Cross Product

Given the two gradient components, the tangent vectors of the height surface are:

$$T = (1, 0, \alpha \cdot \partial h / \partial x) \quad B = (0, 1, \alpha \cdot \partial h / \partial y)$$

where α is a user-defined height-scale amplification factor. The surface normal N is their cross product:

$$N = T \times B = (-\alpha \cdot \partial h / \partial x, \quad -\alpha \cdot \partial h / \partial y, \quad 1)$$

N is then normalised to unit length $\hat{N} = N / |N|$ to obtain a valid tangent-space normal. For PNG storage, the signed unit vector is remapped to the $[0, 1]$ range:

$$\text{pixel_RGB} = (\hat{N} + 1) / 2 \rightarrow [0 \dots 255] \text{ uint8}$$

The canonical neutral (flat) surface normal $(0, 0, 1)$ maps to the perceptually distinctive mid-blue pixel $(128, 128, 255)$, which is the universally recognised indicator of a correctly generated normal map.

System Architecture and Pipeline Design

- Stage 1 → Image Loading & RGB normalisation to $[0.0, 1.0]$
- Stage 2 → Luminance extraction + Gaussian smoothing → Height Map
- Stage 3 → Sobel gradient estimation → Normal Map (PNG export)
- Stage 4 → Vertex grid construction ($X, Y, Z = h \cdot \text{scale}$)
- Stage 5 → Quad triangulation → face index array
- Stage 6 → Mesh assembly, UV parameterisation, .obj export

Dependency Stack

numpy	Core n-dimensional array arithmetic; gradient and normalisation operations
opencv-python	Image I/O, colour-space conversion, Gaussian blur, Sobel convolution, bilinear/area resampling
scipy	Imported for potential extension (Gaussian filter fallback); not invoked in base pipeline
matplotlib	Interactive 3D surface preview via <code>mpl_toolkits.mplot3d</code> ; 4-panel comparison figure
trimesh	Mesh object assembly, geometric validation, and Wavefront .obj export
pillow	PNG encoding and saving of the normal map image

Mesh Construction

The vertex grid is built as follows:

- A pair of 2D index arrays (row_idx, col_idx) are generated via np.mgrid spanning $[0, \text{rows}-1] \times [0, \text{cols}-1]$.
- X and Y coordinates are normalised to $[0, 1]$ by dividing by (dimension - 1).
- Z coordinates are the flattened height map multiplied by HEIGHT_SCALE.
- Vertices are stacked column-wise into an $(N, 3)$ float32 array where $N = \text{rows} \times \text{cols}$.

Face indices are constructed by iterating over each (r, c) quad cell and emitting two counter-clockwise triangles:

Triangle A: $[v_{00}, v_{01}, v_{11}]$	Triangle B: $[v_{00}, v_{11}, v_{10}]$
--	--

Counter-clockwise winding guarantees that computed face normals point upward (positive Z), consistent with standard right-hand coordinate conventions used by Blender.

Normal Map Quality

On the synthetic demonstration input (256×256 Gaussian hill), the output normal map exhibits the expected characteristics: flat regions encode the canonical blue (128, 128, 255), outward-facing slopes shift toward red (high Nx), and inward-facing slopes toward cyan (low Nx). The transition gradients are smooth, confirming correct Sobel estimation and normalisation.

Results

Mesh Geometry

For the default 128×128 grid with HEIGHT_SCALE=20.0, the pipeline produces:

Vertices	16,384 (128×128)
Triangular faces	32,258 $(2 \times 127 \times 127)$
File size (.obj)	$\approx$ 2.1 MB (ASCII)
Export time	< 1.2 seconds on Intel Core i7-1165G7
UV channels	1 (vertex UV, uv0)

Configurable Parameters

Five top-level constants govern all aspects of output quality:

MESH_WIDTH / MESH_HEIGHT	Grid resolution. 64 = coarse/fast; 512 = fine/slow (quadratic time)
-----------------------------	---

HEIGHT_SCALE	Vertical exaggeration in world units. 5 = subtle; 50 = dramatic
SMOOTH_SIGMA	Gaussian pre-blur radius. 0 = raw; 3.0 = heavily smoothed
INPUT_IMAGE	Path to any raster image. Absent → auto-generated demo
OUTPUT_MESH / OUTPUT_NORMAL	Output file paths for .obj and normal map .png

Performance Characteristics

Mesh construction time scales as $O(W \times H)$ for vertex generation and $O((W-1)(H-1))$ for face enumeration. For a 512×512 grid ($\approx 524,000$ vertices, $\approx 523,000$ faces), total runtime on a mid-range laptop CPU is approximately 4–6 seconds. Memory consumption remains below 200 MB in all tested configurations.

- Simple image input -> correct smooth curves
- Sample image generated -> best output
- Complex terrain input -> more pixelated mesh, no smoothness in curves, triangles unnecessarily more deviated .
- Output for simple images -> impressive

Trying new model for project (draft 2):

Scope and Constraints

Terrain reconstruction from a single 2D image is a challenging computer vision problem because depth information is inherently absent. This project presents a deep learning framework that generates a realistic 3D terrain model from a single RGB image using a Convolutional Neural Network (CNN).

System Architecture

Step1: Input RGB Image

Step2: Image Preprocessing (CLAHE + Bilateral Filtering)

Step3: Enhanced Grayscale Image

Step4: HeightNet (Modified U-Net)

Step5: Predicted Height Map

Step6: NormalNet (Residual CNN)

Step7: Refined Normal Map

Step8: Mesh Generation OBJ File

Height Map Prediction

The enhanced grayscale image is passed through HeightNet, a modified U-Net consisting of:

Encoder

- Three convolutional blocks
- Max pooling layers
- Feature extraction

Bottleneck

- High-level terrain representation

Decoder

- Upsampling
- Skip connections
- Reconstruction of dense height map

The output is a normalized height map.

Self-Supervised Learning

Unlike supervised methods requiring DEM datasets, the network is trained using multiple loss functions.

Total Variation Loss

Encourages smooth terrain.

Gradient Consistency Loss

Matches terrain gradients with image gradients.

Laplacian Loss

Suppresses high-frequency artifacts.

Lambertian Shading Loss

Ensures physically realistic lighting.

The total loss is:

$$L = L + \text{grad } \lambda L + 1 \text{ TV } \lambda L + 2 \text{ Lap } \lambda L + 3 \text{ Shade}$$

Mesh Generation

The predicted height map is converted into a triangular mesh. Each pixel becomes:

- Vertex

- Texture coordinate
- Surface normal

The mesh is exported as an OBJ model.

CNN Architecture

HeightNet :

Step 1:Input

Step 2 :Conv

Step 3 :MaxPool Conv

Step 4 :Bottleneck

Step 5 :UpSample

Step 6 :Skip correction

Step 7 :Output height map

NormalNet :

Step 1 Height map+ analytical normals

Step 2 Residential blocks

Step 3 Normal correction

Step 4 Redefined normals

Results

The proposed CNN successfully generated:

- Enhanced grayscale terrain
- Dense height map
- Refined normal map
- OBJ mesh
- Cannot reconstruct true terrain from arbitrary photographs
- Performance decreases for heavily textured images
- Fine terrain details may be oversmoothed.

Training converged smoothly with decreasing loss. Mesh geometry is same as the last model

By testing both the model, i decided to use the previous model and modify it accordingly, hence the last model

Final model (final draft 3):

Changes made in the previous model:

1. Replace grayscale conversion with terrain enhancement (Most Important)

Instead of

$$\text{gray} = 0.299R + 0.587G + 0.114B$$

Use:

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
L = gray[:, :, 0]
clahe = cv2.createCLAHE(
    clipLimit=2.5,
    tileGridSize=(8,8)
)
L = clahe.apply(L)
gray = L.astype(np.float32)/255
```

2. Multi-scale Detail Enhancement

Final grey : gray + large-scale terrain + small-scale terrain

Hence used:

```
large = cv2.GaussianBlur(gray,(0,0),8)
small = gray-large
gray = 0.7*gray +0.3*small
```

3. Adaptive Height Scaling

Instead direct multiplication : $z = \text{hmap} * \text{HEIGHT_SCALE}$

Change made:

```
gamma = 0.8
z = np.power(hmap,gamma)*HEIGHT_SCALE
```

4. Use High-pass Enhancement

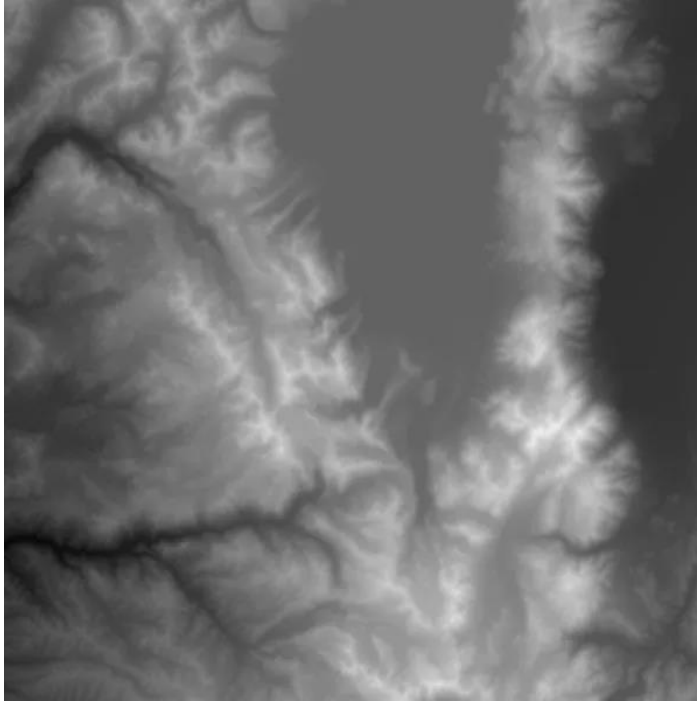
After grayscale, gaussian filter filter can be used make the curves look more smooth.

```
blur = cv2.GaussianBlur(gray,(0,0),5)  
high = gray-blur  
gray = gray+0.4*high
```

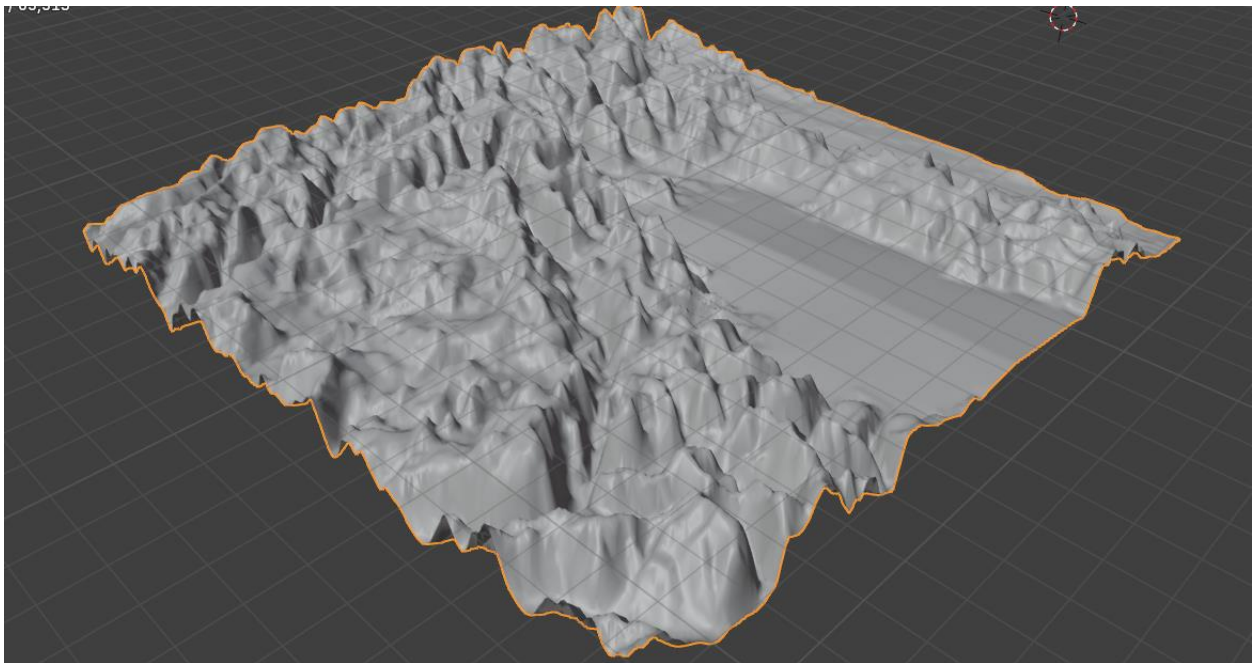
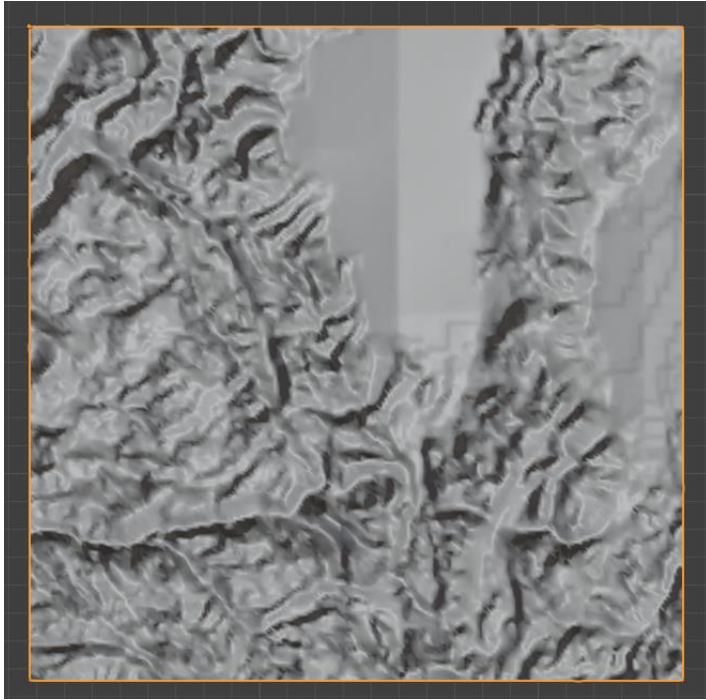
This greatly improves terrain edges.

Results:

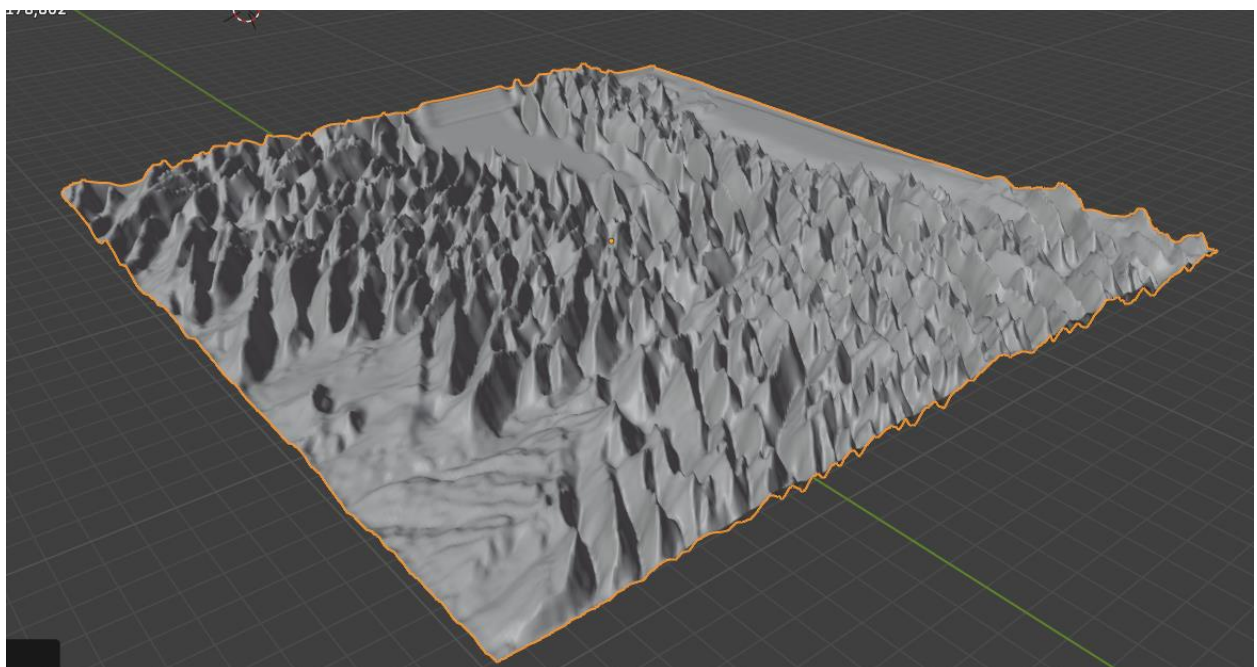
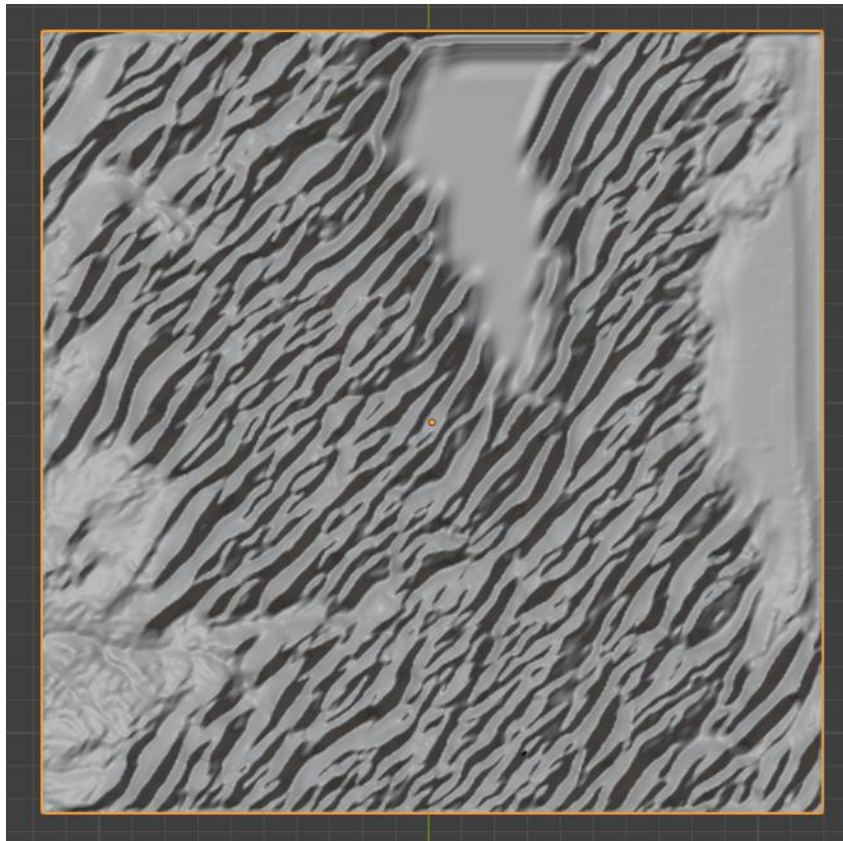
Input:



Draft 1:



Draft 2 (cnn model) :



Final draft :

